Applied Machine Learning Lab

B.Tech Semester 4

Experiment 05

Stock Price Prediction using Regression Models

Date: 11th feb 2026

SAP ID: 590016154

Name: Vidisha

AIM

To develop and evaluate regression-based machine learning models for predicting stock prices using historical market data.

THEORY

Stock price prediction is a regression problem where future values are estimated using historical data.
Time series features like lag values and moving averages are used.
Models include Linear, Ridge, Lasso, SVR and Random Forest.
Evaluation metrics include MAE, MSE, RMSE and R2.

Commands/Code Used

```
# Import libraries

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns


from sklearn.linear_model import LinearRegression, Ridge, Lasso

from sklearn.svm import SVR

from sklearn.ensemble import RandomForestRegressor

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score


# Load dataset

df = pd.read_csv("Tesla.csv")


# Feature Engineering

df["Prev_Close"] = df["Close"].shift(1)

df["MA_5"] = df["Close"].rolling(5).mean()

df["MA_10"] = df["Close"].rolling(10).mean()


# Target variable

df["Target_Close"] = df["Close"].shift(-1)


# Drop missing values

df = df.dropna()
```

```
# Features & target
X = df.drop(["Target_Close"], axis=1)
y = df["Target_Close"]

# Train-test split (time-based)
split = int(len(df) * 0.8)
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

# Models
models = {
   "Linear Regression": LinearRegression(),
   "Ridge": Ridge(),
   "Lasso": Lasso(),
   "SVR": SVR(),
   "Random Forest": RandomForestRegressor()
}

# Store results
results = {}

# Train and evaluate
for name, model in models.items():
   model.fit(X_train, y_train)
   y_pred = model.predict(X_test)
```

```
    mae = mean_absolute_error(y_test, y_pred)
    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test, y_pred)

    results[name] = [mae, mse, rmse, r2]

    print(f"\n{name}")
    print("MAE:", mae)
    print("MSE:", mse)
    print("RMSE:", rmse)
    print("R2:", r2)

# Convert results to DataFrame
results_df = pd.DataFrame(results, index=["MAE", "MSE", "RMSE", "R2"]).T

# Plot model comparison
results_df.plot(kind="bar", figsize=(10,6))
plt.title("Model Comparison")
plt.xticks(rotation=45)
plt.show()

# Stock price trend
plt.figure(figsize=(10,5))
plt.plot(df["Close"])
plt.title("Tesla Stock Closing Price")
```

```
plt.show()

# Correlation heatmap
plt.figure(figsize=(8,6))
sns.heatmap(df.corr(), annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()

# Feature importance (Random Forest)
rf = RandomForestRegressor()
rf.fit(X_train, y_train)

importances = rf.feature_importances_
features = X.columns

importance_df = pd.DataFrame({"Feature": features, "Importance": importances})
importance_df = importance_df.sort_values(by="Importance")

plt.figure(figsize=(8,6))
plt.barh(importance_df["Feature"], importance_df["Importance"])
plt.title("Feature Importance")
plt.show()

# Actual vs Predicted (using best model, e.g., Lasso)
model = Lasso()
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)


plt.figure(figsize=(6,6))

plt.scatter(y_test, y_pred)

plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--')

plt.xlabel("Actual")

plt.ylabel("Predicted")

plt.title("Actual vs Predicted")

plt.show()


# Residual analysis

residuals = y_test - y_pred


plt.figure(figsize=(12,5))


plt.subplot(1,2,1)

plt.scatter(y_pred, residuals)

plt.axhline(0, color='red')

plt.title("Residuals vs Predicted")


plt.subplot(1,2,2)

sns.histplot(residuals, kde=True)

plt.title("Residual Distribution")


plt.show()
```

RESULTS

Stock price trends visualized.
Correlation heatmap generated.

Feature importance analyzed.
Model comparison performed.
Actual vs predicted values plotted.
Residual analysis done.

CONCLUSION

Regression models successfully predicted stock prices. Feature engineering improved performance.